

GraphCogent: Mitigating LLMs' Working Memory Constraints via Multi-Agent Collaboration in Complex Graph Understanding

Rongzheng Wang

Institute of Intelligent Computing,
University of Electronic Science and
Technology of China
Chengdu, China
wangrongzheng@std.uestc.edu.cn

Shuang Liang^{*†}

Institute of Intelligent Computing,
University of Electronic Science and
Technology of China
Chengdu, China
shuangliang@uestc.edu.cn

Qizhi Chen

School of Computer Science and
Engineering, University of Electronic
Science and Technology of China
Chengdu, China
chenqizhi@std.uestc.edu.cn

Yihong Huang

Institute of Intelligent Computing,
University of Electronic Science and
Technology of China
Chengdu, China
2021080910004@std.uestc.edu.cn

Muquan Li

School of Information and Software
Engineering, University of Electronic
Science and Technology of China
Chengdu, China
muquanli2023@std.uestc.edu.cn

Yizhuo Ma

Institute of Intelligent Computing,
University of Electronic Science and
Technology of China
Chengdu, China
myz@std.uestc.edu.cn

Dongyang Zhang[†]

Institute of Intelligent Computing,
University of Electronic Science and
Technology of China
Chengdu, China
dyzhang@uestc.edu.cn

Ke Qin[†]

School of Computer Science and
Engineering, University of Electronic
Science and Technology of China
Chengdu, China
qinke@uestc.edu.cn

Man-Fai Leung

School of Computing and Information
Science, Anglia Ruskin University
Cambridge, United Kingdom
man-fai.leung@aru.ac.uk

Abstract

Large language models (LLMs) show promising performance on small-scale graph reasoning tasks but fail when handling real-world graphs with complex queries. This phenomenon arises from LLMs' working memory constraints, which result in their inability to retain long-range graph topology over extended contexts while sustaining coherent multi-step reasoning. However, real-world graphs are often structurally complex, such as Web, Transportation, Social, and Citation networks. To address these limitations, we propose **GraphCogent**, a collaborative agent framework inspired by human *Working Memory Model* that decomposes graph reasoning into specialized cognitive processes: sense, buffer, and execute. The framework consists of three modules: Sensory Module standardizes diverse graph text representations via subgraph sampling. Buffer Module integrates and indexes graph data across multiple formats, and Execution Module combines tool calling and tool creation for efficient reasoning. We also introduce Graph4real, a comprehensive benchmark that contains four domains of real-world graphs (Web, Transportation, Social, and Citation) to evaluate LLMs' graph reasoning capabilities. Our Graph4real covers 21 different graph

reasoning tasks, categorized into three types (Structural Querying, Algorithmic Reasoning, and Predictive Modeling tasks), with graph scales up to 10 times larger than existing benchmarks. Experiments show that Llama3.1-8B based GraphCogent achieves a 50% improvement over massive-scale LLMs like DeepSeek-R1 (671B). Compared to state-of-the-art code-based baseline, our framework outperforms by 20% in accuracy while reducing token usage by 80% for in-toolset tasks and 30% for out-toolset tasks.

CCS Concepts

• **Networks** → **Network algorithms**; • **Computing methodologies** → **Knowledge representation and reasoning**; • **Mathematics of computing** → **Graph algorithms**.

Keywords

Large Language Model, Graph Algorithms, Multi-agent System

ACM Reference Format:

Rongzheng Wang, Shuang Liang, Qizhi Chen, Yihong Huang, Muquan Li, Yizhuo Ma, Dongyang Zhang, Ke Qin, and Man-Fai Leung. 2026. GraphCogent: Mitigating LLMs' Working Memory Constraints via Multi-Agent Collaboration in Complex Graph Understanding. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792314>

^{*}Corresponding author.

[†]Also with Ubiquitous Intelligence and Trusted Services Key Laboratory of Sichuan Province.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792314>

1 Introduction

Large language models (LLMs) [30] have demonstrated remarkable cognitive capabilities in natural language processing. However, when querying the connection path between two webpages in

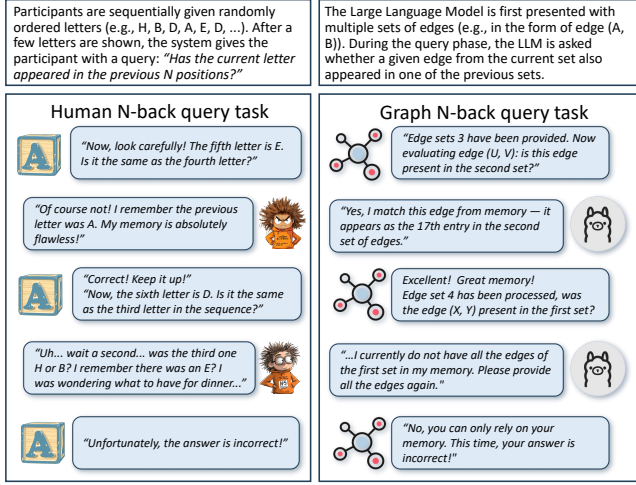


Figure 1: Graph N-back Query Task: A graph is split into 50-edge subsets E_t . At turn $t + N$, the LLM verifies edge existence in E_t . Experimental results are in Section 4.1.

a Web graph, state-of-the-art LLMs like DeepSeek-R1 [8], GPT-o3 [30], and Gemini-2.5 pro [10] return incorrect navigation routes in 9 out of 10 cases. This failure reveals that current LLMs remain limited in handling large real-world graph reasoning tasks.

Researchers have explored various methods to address the constraints of LLMs' inability to perform graph reasoning. *Text-based* methods [7, 9, 25] leverage Chain-of-Thought (CoT) [40] reasoning but struggle with the dynamic programming requirements of graph algorithms (e.g., Bellman-Ford [5]), resulting in cascading errors in complex tasks. *Tool-based* methods [39, 43] rely on external tools for computation, requiring predefined tools and rigid input formats (e.g., preprocessed files). These limitations hinder their ability to process diverse graph text representations, such as adjacency lists, symbolic notations, and linguistic descriptions [9], thereby reducing their adaptability to real-world scenarios. *Code-based* methods [23, 44] employ LLMs' code generation capability, often decomposing graph tasks into sequential stages via naive instruction prompts. However, these approaches are limited by the complexity of code construction and the inherent memory constraints of LLMs. As graph scales increase and tasks involve real-world graph problems, code-based methods perform poorly when handling graph data comprehension and task execution.

This limitation stems from LLMs' working memory constraints that restrict simultaneous processing of complex graph topology and multi-step reasoning. A common mitigation approach involves partitioning graphs into smaller units for sequential processing [17], mirroring how humans decompose complex problems into manageable information chunks [28]. Classic human working memory capacity assessments N-back test [1, 18, 19] (illustrated in Fig. 1, left) reveals a rapid cognitive decline when humans are required to retain more than three items. Inspired by the N-back test, we design an analogous LLMs' Graph N-back test. As illustrated in the right of the Fig. 1, by the 3rd processing turn, Llama3.1-8B [36] exhibits significant topology forgetting. This demonstrates that LLM graph

reasoning similarly suffers from working memory constraints, mirroring human cognitive limitations.

Human brain addresses these constraints through a specialized working memory model for sensory processing, information buffering, and task execution [4]. Facing a complex task, humans rely on an external sensory system to perceive stimuli, an episodic buffer to integrate and store information, and a central executive to coordinate cognition and processing. This absent architecture in monolithic LLMs reveals their key graph reasoning bottlenecks: (1) *Diverse Text Representations*: The sensory limitation in processing diverse graph text representations leads to inconsistent comprehension, especially for mainstream open-source models (e.g., Llama [36]), thus impairing reasoning accuracy; (2) *Overload Graph Scale*: Lacking a buffer mechanism, LLMs' limited input context windows hinders long-range dependency capture, causing global information loss and cascading reasoning errors; (3) *Code Execution Fragility*: The executive dysfunction results in unreliable code generation and inefficient algorithmic implementation.

Inspired by the architecture of human working memory model, we propose **GraphCogent**, a **Graph Collaboration Agentic** framework designed to mitigate LLMs' working memory constraints in graph reasoning. Our framework consists of three modules: **Sensory Module** employs a sensory agent to sample subgraph and transform unstructured graph text representations into standardized adjacency list. **Buffer Module** integrates graph data from the Sensory Module and establishes data indices for data formats (including NumPy, PyG, and NetworkX) based on Buffer Agent. **Execution Module** combines tool calling and on-demand tool creation. Naive graph reasoning tasks are handled by a Execution Agent through a pre-built common toolset, while complex tasks leverage a tool creator that produces modular, task-specific components and backfills new tool requests with the Buffer's preprocessed data, thereby avoiding error-prone full-code generation.

To evaluate LLM's generalization capability in graph reasoning, we construct a more challenging Graph4real benchmark dataset, which collects graph data from four real-world domains: Web [20], Social [27], Transportation [6], and Citation [41]. Our Graph4real covers 21 graph reasoning tasks, categorized into three types (Structural Querying, Algorithmic Reasoning, and Predictive Modeling tasks), with graph scales that are 10 times larger than existing benchmarks (e.g., NLGraph [37], GraphWiz [7]). Experimental results demonstrate that our method achieves state-of-the-art performance with an average accuracy of 98.5% based on the Llama3.1-8B, representing a 20% improvement over other code-based approaches. Cross-dataset validation demonstrates robust generalization, maintaining over 90% accuracy across diverse public benchmarks. The combined tool calling and tool creation strategy delivers dual optimization: it reduces token consumption by 80% for tasks covered by our toolset while still achieving 30% token savings for out-toolset tasks compared to the conventional code-based method.

Our contributions are summarized as follows: (1) We propose GraphCogent, a novel agent collaboration framework addressing LLMs' working memory constraints through its sensory-buffer-execution architecture; (2) We design a reasoning mechanism that combines tool calling and tool creation with a Thinking Policy Initialization to stabilize early-stage decision-making and a tailored Capability-Margin Preference Optimization (CMPO) objective to

sharpen tool discrimination, enabling efficient handling of diverse graph tasks; (3) We construct Graph4real, a comprehensive benchmark featuring real-world graphs that covers 21 different graph reasoning tasks, categorized into three types (Structural Querying, Algorithmic Reasoning, and Predictive Modeling tasks), with graph scales that are 10 times larger than existing benchmarks.

2 Related Work

LLMs for Complex Reasoning Tasks. LLM-based reasoning faces increasing challenges as input complexity grows. In pure-text scenarios, while step-by-step prompting aids derivation, such traces often become brittle and accumulate errors as reasoning depth increases [9, 13, 25, 38]. Transitioning to structured graph data requires models to handle diverse text representations while maintaining topological context; however, current approaches encounter scalability bottlenecks when processing complex relational structures [7, 26, 30, 39, 43]. Further introducing multimodal and temporally evolving signals brings additional challenges in cross-modal alignment and context management, which further stress reasoning robustness and memory capacity [24, 33, 34].

Working Memory Constraints in LLMs. Like humans facing working memory constraints in complex tasks [15, 29], LLMs show capacity constraints. Cognitive studies show humans typically retain only few information units simultaneously, with performance decaying in N-back tests (where $N = 3$) [1, 16, 18, 19]. LLMs mirror this pattern, exhibiting accuracy declines on 3-step dependencies [3]. This working memory overload manifests in graph reasoning as excessive graph scale and complex reasoning tasks, thus hindering the processing of intricate graph structures and multi-step reasoning, which has motivated complementary attention and retrieval enhancements for long-context reasoning [21, 22].

Graph Reasoning Benchmarks for LLMs. NLGraph [37], as the pioneering benchmark, demonstrates LLMs' competence on small-scale graphs (less than 40 nodes) with basic reasoning tasks. GraphWiz [7] later expands to 100 nodes with more complex tasks. However, these and other benchmarks [9, 25, 38, 39] remain constrained by fixed-size graphs within context windows and templated reasoning tasks, failing to match real-world reasoning demands where both scale and complexity exceed current test conditions.

3 Graph4real Construction

Current LLMs' graph reasoning benchmarks face three key limitations: (1) Limited scale, primarily using randomly generated graphs that lack real-world topological characteristics; (2) Simple textual representations, over-relying on preprocessed files or adjacency lists while ignoring domain-specific semantic descriptions; (3) Artificial task formulations, where queries directly specify algorithms with only a single sentence rather than mirroring real-world reasoning scenarios requiring intent interpretation.

Real-World Graph Scaling. To establish a comprehensive benchmark for evaluating LLMs on real-world graph reasoning tasks, we curate a diverse set of graphs from four domains: Web (Google Web Graph [20]), Social (SNAP Social Circles [27]), Transportation (PeMS [6]), and Citation graphs (Cora [41]). Unlike prior benchmarks [25, 37] that rely on randomly generated graphs, our dataset

is constructed from real-world sources to ensure realistic topological properties and domain-specific characteristics. We employ a biased random walk sampling strategy to construct graphs at three scales (40, 100, and 1000 nodes), ensuring both effective evaluation of LLM-based methods through smaller-scale graphs and construct large-scale graphs to challenge existing methods.

Various Graph Text Representation. We formalize three graph text representation formats to evaluate LLMs' graph comprehension and reasoning capabilities: (1) Adjacency list ($[[0,1],[0,2],...]$); (2) Symbolic notation ($0 \rightarrow 1, 2 \rightarrow 3$); (3) Linguistic descriptions using domain-specific predicates (Linked/Followed/Connected/Cited). These representations preserve various semantics, enabling assessment of LLMs' ability to process both formal graph formats and domain-specific descriptions.

Intent-Driven Task Design. A total number of 21 tasks are designed and categorized into three classes: (1) *Graph Structural Querying Tasks*: Focus on fundamental graph properties, such as edge existence or node count, testing basic graph structural comprehension. (2) *Graph Algorithmic Reasoning Tasks*: Require reasoning based on classical algorithms, such as shortest path and maximum flow, evaluating algorithmic proficiency. (3) *Graph Predictive Modeling Tasks*: Involve neural network-based predictions, such as node classification and link prediction, assessing predictive capabilities.

For task generation, we collect five real-world scenarios for each of the four domains (e.g., travel planning and logistics optimization for transportation) and craft 20 prompt templates to construct contextually grounded questions. Using DeepSeek-R1 [8], we generate graph reasoning tasks based on templates with specific scenarios, ensuring both diversity and relevance. To validate task quality, we employ a dual evaluation approach: DeepSeek-R1 for automated assessment and human for manual review. This process produces a dataset with 4200 questions distributed across the 21 tasks. We enumerate the 21 defined tasks in Appendix E.

4 GraphCogent

We propose GraphCogent, a framework inspired by the human working memory model to address the challenges of graph reasoning tasks. Our framework consists of three modules: Sensory Module (samples subgraphs and transforms unstructured graph text representations into standardized adjacency list), Buffer Module (integrates graph data from Sensory Module and establishes data indices for data formats), and Execution Module (combines tool calling and tool creation to address graph reasoning tasks). The pipeline of GraphCogent is illustrated in Fig. 2.

4.1 Graph N-back Query

Given large graph scale and varied text representation in real-world settings, we test whether LLMs can retain edge information across dialogue turns. We propose Graph N-back Query, inspired by the human working memory N-back task [18]. Using a 100-node graph sampled from PeMS datasets [6], edges are partitioned into 50-edge subsets E_t , and edge existence is queried at turn $t + N$ under three text representations. As shown in Fig. 3a, experiments with Llama3.1-8B exhibit substantial forgetting across dialogues, indicating that LLMs fail to reliably maintain global graph information

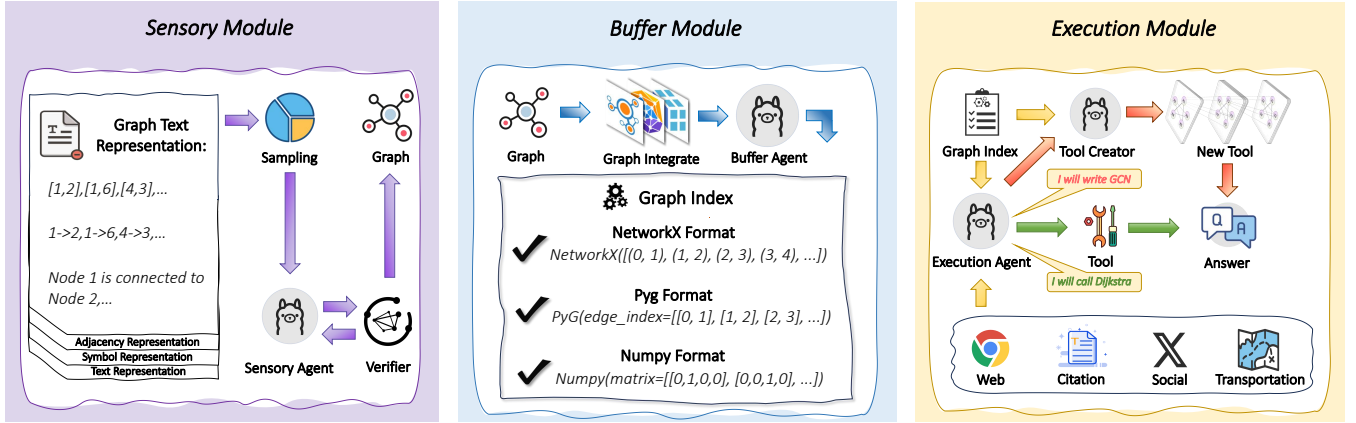


Figure 2: Overview of GraphCogent. Sensory Module (left) standardizes various graph text representations through subgraph sampling and conversion; Buffer Module (center) establishes cross-format data (e.g., NetworkX) integrating and indexing transformations; Execution Module (right) enables two reasoning modes: Execution Agent is employed for tool discrimination and implements tool calling for in-toolset tasks, Tool Creator handles out-toolset tasks based on tool creation.

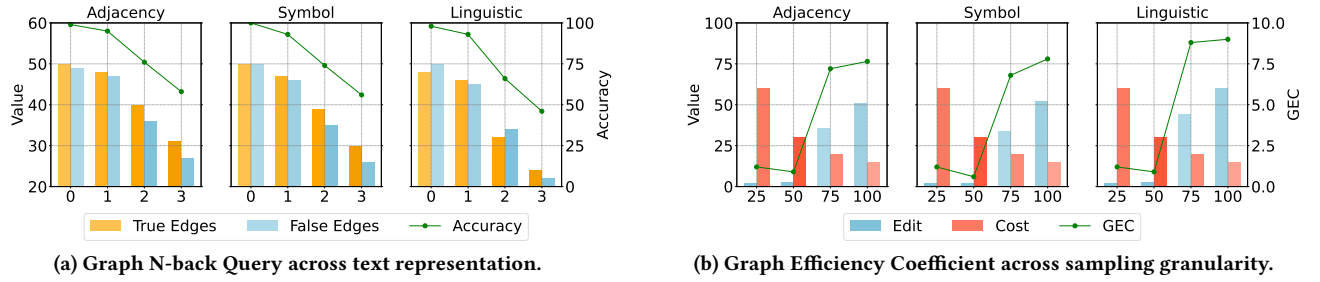


Figure 3: Memory retention and efficiency under different graph representations. (a) Accuracy is computed as True-edge and False-edge correctness; (b) Edit is graph edit distance and Cost (scaled $\times 100$) is total LLM interactions; lower GEC is better.

for large-scale reasoning. This suggests that context window limitations act as a working memory bottleneck, making conventional step-by-step graph input ineffective under these constraints.

4.2 Sensory Module

The Sensory Module, which is proposed to address the dual challenges of graph scale and text representations, consists of two key components: Sensory Agent parses raw graph data via subgraph sampling and transforms diverse text representations inputs into standardized adjacency lists; Graph Verifier serves as a supervisory mechanism for the Sensory Agent’s transformation process, detecting both format deviations and misaligned conversions.

Sensory Agent: Experiments in Fig. 3b demonstrate that the granularity of sampling directly impacts LLMs’ transformation performance. To achieve accurate text representation transformation, we define the Graph Efficiency Coefficient (GEC) to balance transformation accuracy (measured by Edit distance) against computational Cost (LLM token consumption):

$$\text{GEC} = \text{Edit_distance}(G, G') \times \frac{\sum_{i=1}^N (T_{\text{input}}^i + T_{\text{output}}^i)}{T_{\text{max}}}, \quad (1)$$

where $\text{Edit_distance}(G, G')$ measures the structural deviation between the original graph G and the sampled graph G' ; $\sum_{i=1}^N (T_{\text{input}}^i + T_{\text{output}}^i)$ quantifies the total token cost of LLM interactions, normalized by T_{max} (maximum allowed tokens per LLM interaction). The algorithm for calculating Edit_distance and the rationale for selecting token count as the Cost metric are detailed in Appendix C.

We evaluate GEC on 100 graphs (300 edges each) under four subgraph granularities (25–100 edges) using Llama3.1-8B by measuring Edit distance and token Cost. Fig. 3b shows a clear trade-off: coarser windows increase Edit distance due to transformation failures, while finer windows raise Cost due to more LLM calls. For Llama3.1-8B the best GEC occurs at around 50 edges (model-dependent across architectures and scales). Guided by this criterion, the Sensory Agent partitions text-represented graphs with a GEC-selected sliding window (default 50 edges) and a fixed 5% overlap, uses heuristic prompts to normalize varied input formats into standardized adjacency lists, parses weighted and unweighted outputs with two regex templates, and validates extracted edges with a Graph Verifier. For structured graph files (e.g., edgelist, txt, json), it skips windowing and directly loads them into the Buffer Module.

Graph Verifier. While GEC controls the granularity, LLM outputs can still introduce parsing noise during transformation. The

Verifier checks quantity consistency ($|V|$, $|E|$) and adjacency-list format compliance via regex parsing, and failed transformations trigger Sensory Agent retries. After per-window transformation, the Verifier merges subgraphs via a deterministic set-union with de-duplication and then runs a quick quantity-and-format check to ensure global consistency before dispatching any reasoning. By default, text graphs use a 50-edge window with 5% overlap, and file-based graphs are not partitioned. Experiments on how the Graph Verifier enhances transformation reliability and ensures global consistency are provided in Appendix B.1.

4.3 Buffer Module

Current frameworks for graph reasoning lack storage mechanisms, relying instead on pre-processed files or delegating preprocessing to code generation. As Fig. 3b shows, even adjacency list transformations exhibit growing Edit distance with scale increasing, revealing LLMs' declining reliability in graph comprehension. Moreover, graph paradigms diverge in storage: algorithm tasks use OOP structures (e.g., NetworkX), link prediction requires tensor formats (e.g., PyG), and traffic prediction needs sparse matrices (e.g., NumPy). This difference overloads LLMs' working memory, resulting in inaccurate code generation, demanding both task-accurate implementations and cross-format transformation via reasoning.

Based on the human episodic buffer mechanism [12], we construct the Buffer Module to serve as an intermediary that not only stores and integrates graph data but also establishes indices for different data formats. This module transforms raw graph text representations into multiple standardized formats: NetworkX for graph algorithms, NumPy for numerical operations, and PyG for tensor requirement tasks.

The Buffer Module begins by integrating adjacency lists from the Sensory Module to construct graph structures. To support diverse reasoning tasks, the graph data is transformed into standardized data formats. For Structural Querying and Algorithmic Reasoning tasks, the data is preprocessed into NetworkX graph objects that preserve topological relationships and node attributes. For Predictive Modeling tasks, we implement data preprocessing including normalization and outlier treatment before encapsulating the graph as NumPy arrays or PyG tensors. To enable efficient access across these data formats, we implement the Buffer Agent to build data index that tracks key characteristics including data dimensionality, organizational schema, and metadata description. These indices are then passed to the Execution Module, allowing it to rapidly retrieve the most suitable graph format for each specific reasoning task.

4.4 Execution Module

Execution Module addresses code construction complexity and inefficiency with two components: an Execution Agent and a Tool Creator. The Execution Agent analyzes each task and invokes a suitable tool from a common toolset when the task is covered. Otherwise, it requests the Tool Creator to synthesize a minimal task-specific tool to fill the pending invocation, leveraging Buffer-preprocessed data to avoid brittle full-code generation while retaining adaptability to complex requirements. Newly created tools can be registered into the toolset for future reuse.

4.4.1 Execution Agent. We adopt a two-stage instruction-tuning pipeline that couples (i) Thinking Policy Initialization for robust tool invocation with (ii) Capability-Margin Preference Optimization (CMPO) for reliable tool capability discrimination. The first stage follows instruction-tuned policy learning with explicit thinking traces [14]; the second stage follows preference-based policy learning [32] to shape a likelihood-ratio margin around the tool capability boundary.

Phase 1: Thinking Policy Initialization for Tool Invocation.

We pair graph reasoning problems $Prob$ with toolset descriptions $Toolset$ to form queries $Ques = \langle Prob, Toolset \rangle$. These are processed by DeepSeek-R1 to produce $Ans = \langle Think, Tool \rangle$, where $Think$ exposes the reasoning trace to enhance routing stability and auditability, and $Tool$ indicates the selected tool. We curate pairs with correct invocation and optimize the maximum-likelihood objective:

$$\mathcal{L}_{init} = - \sum_{i=1}^N \log P_{\theta}(\langle Think, Tool \rangle_i | \langle Prob, Toolset \rangle_i), \quad (2)$$

with the factorization:

$$\log P_{\theta}(Ans | Ques) = \underbrace{\log P_{\theta}(Think | Ques)}_{\text{thinking generation}} + \underbrace{\log P_{\theta}(Tool | Think, Ques)}_{\text{tool selection}}. \quad (3)$$

While the first stage provides invocation priors, Appendix B.3 shows a boundary effect: the agent may still over-rely on existing tools for tasks that require creation. This motivates a decision margin for the agent's tool capability. For later unification, we define a policy score relative to a frozen reference policy P_{ref} for subsequent preference optimization and tool capability discrimination:

$$s_{\theta}(y | x) = \log P_{\theta}(y | x) - \log P_{ref}(y | x). \quad (4)$$

Phase 2: Capability-Margin Preference Optimization.

To refine agent's tool capability discrimination, we adopt following two strategies.

(a) *Action-segment scoring.* $\mathcal{M}_{tool}$ denotes the token indices belonging to the action segment (demarcated by special tool markers in data). We compute a policy score centered at a frozen reference policy P_{ref} but restricted to the action segment:

$$s_{\theta}^{tool}(y | x) = \sum_{t \in \mathcal{M}_{tool}} \log P_{\theta}(y_t | y_{<t}, x) - \sum_{t \in \mathcal{M}_{tool}} \log P_{ref}(y_t | y_{<t}, x). \quad (5)$$

The score aggregates the decision-critical tokens within the tool segment, isolating action selection from reasoning-style variance. We then quantify the preference between a preferred response R_w and a less-preferred response R_l by a likelihood-ratio margin:

$$\Delta s^{tool} = s_{\theta}^{tool}(R_w | x) - s_{\theta}^{tool}(R_l | x). \quad (6)$$

(b) *Capability-aware hard negatives.* For each x , we sample a candidate set $C_K(x)$ from the initialized policy and select a challenging negative via the reference action-segment likelihood:

$$R_l = \underset{y \in C_K(x) \setminus \{R_w\}}{\operatorname{argmax}} \ell_{ref}^{tool}(y | x), \quad (7)$$

$$\ell_{ref}^{tool}(y | x) = \sum_{t \in \mathcal{M}_{tool}} \log P_{ref}(y_t | y_{<t}, x). \quad (8)$$

This selection aims to identify the most confusing incorrect answer for the agent based on the reference likelihood, forcing the agent to learn to distinguish fine-grained differences between them.

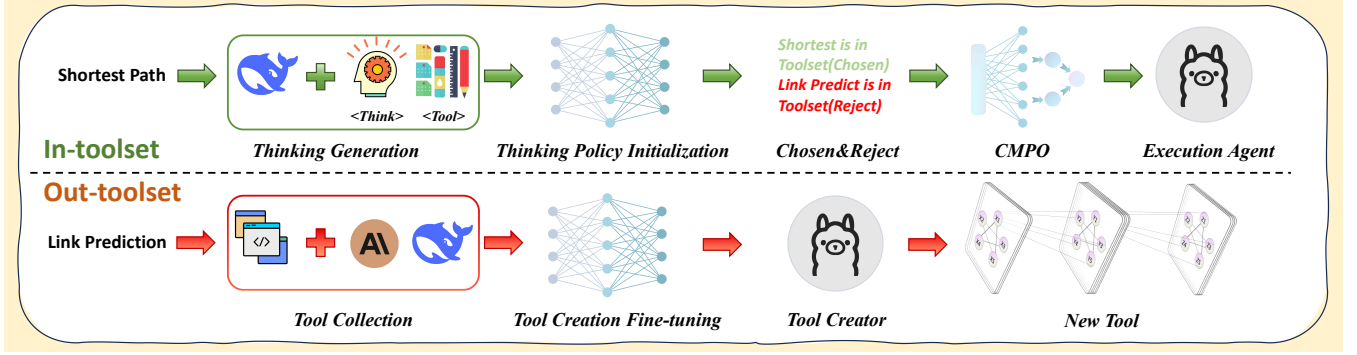


Figure 4: Overview of Execution Agent and Tool Creator training. In-toolset task (top) initializes a thinking policy from Think and Tool pairs, applies a CMPO method to refine Execution Agent’s tool capability discrimination; Out-toolset task (bottom) uses the fine-tuned Tool Creator to synthesize minimal task-specific tools, and backfills them into the common toolset.

We then optimize a logistic objective with a fixed margin m :

$$\mathcal{L}_{\text{CMPO}}^{(\text{margin})} = -\mathbb{E}_{(x, R_w, R_l)} \left[\log \sigma(\beta (\Delta s^{\text{tool}} - m)) \right], \quad (9)$$

which encourages the agent not only to prefer the correct tool over the negative, but to maintain a minimum confidence gap between them, thereby improving robustness near the decision boundary.

At execution time, we define a capability score $\gamma(x) = \Delta s^{\text{tool}}$ and employ a two-threshold hysteresis rule to stabilize tool capability discrimination near the boundary, thereby further ensuring the stability and robustness of the agent’s decisions:

$$\text{route}(x) = \begin{cases} \mathcal{A}_{\text{invoke}}, & \gamma(x) \geq \tau_{\text{high}}, \\ \mathcal{A}_{\text{create}}, & \gamma(x) \leq \tau_{\text{low}}, \\ \text{keep previous decision}, & \text{otherwise.} \end{cases} \quad (10)$$

4.4.2 Tool Creator. The Tool Creator handles missing capability cases by generating only the tool body under a prompt template and returning it to fill the pending invocation. The template supplies the task description, admissible libraries and variable names, and the expected return object; outer interface and skeleton code is provided by the framework at runtime. By executing directly on the Buffer Module’s preprocessed data structures, this design keeps generation reliable and avoids brittle full-file synthesis.

We curate a corpus from classical graph algorithms (e.g., PageRank) and neural models (e.g., GCN, GAT) on github. Using LLMs (DeepSeek-v3 [8], GPT-4o [30], Claude-3.7 [2]), we synthesize code and retain only samples that execute successfully. For each retained sample, we extract the body segment between special body marker. On the input side, we concatenate the original query with its task template to form $\text{Ques}^{\text{Tpl}} = \langle \text{Ques}, \mathcal{T} \rangle$, and keep a minimal contextual scaffold c_i (e.g., necessary imports, a function signature, lightweight placeholders) to induce body-only completion.

Tool Creation Fine-tuning. We optimize a body-only maximum likelihood objective:

$$\mathcal{L}_{\text{train}} = - \sum_{i=1}^N \sum_{t=1}^{M_{\text{body},i}} \log P_{\theta}(y_{i,t} | y_{i,<t}, \text{Ques}_i^{\text{Tpl}}, c_i), \quad (11)$$

where N is the number of validated examples, $M_{\text{body},i}$ denotes the number of tokens in the body segment of example i (we reindex body tokens as $t = 1, \dots, M_{\text{body},i}$), $y_{i,t}$ is the t -th target token within the body. This objective focuses learning on the implementation logic while leaving interface details to the framework. At inference, the template provides the necessary task and contextual cues; the Tool Creator emits the body segment to satisfy the Execution Agent’s pending invocation, which the framework assembles with the runtime skeleton and executes to produce final results.

5 Experiments

5.1 Experimental Settings

Dataset. We evaluate on Graph4real, a real-world benchmark with graphs of 40 to 1,000 nodes from transportation, social, web, and citation domains. It contains both in-toolset and out-toolset tasks, with 500 test instances per task and scale across domains. For fair comparison, we use standard adjacency lists as the graph text representation in the main experiments (Table 1). We also evaluate on public benchmarks: NLGraph [37], Talk like a Graph [9], GraphWiz [7], GraphArena [35], GraCoRe [42], and GraphInstruct [25].

Baseline Methods. We compare three types of methods: (1) *Text-based methods* including GPT-4o [30], Claude-3.7 [2], and DeepSeek-R1 [8] with 2-shot CoT prompting. (2) *Tool-based methods* including Graph-Toolformer [43], GraphTool-Instruction [39] on Llama3.1-8B [36] and GPT-4o with Function Calling [30]; (3) *Code-based methods* are the state-of-the-art GraphTeam [23] framework with GPT-4o-mini [30] and reproduced GCoder [44] with Llama3.1-8b [36]. **Evaluation Metrics.** We use accuracy for all tasks except traffic flow prediction (measured by MAE).

5.2 Experimental Results

5.2.1 Main Results. The performance results are reported in Table 1. Our method outperforms all baseline methods across toolset-covered tasks. As for text-based methods, they demonstrate advantages on small-scale Structural Querying tasks (e.g., DeepSeek-R1 achieves 100% on edge existence). However, these methods show significant performance degradation when dealing with problems

Table 1: Performances of GraphCogent and other baselines on Graph4real in-toolset tasks. Note: Graph-TF, GraphTool-Ins, and GPT4o-FC are abbreviations for Graph-Toolformer, GraphTool-Instruction, and GPT4o-Function Calling.

Type	Method	Edge Existence	Edge Count	Triangle Count	Shortest Path	Path Existence	Cycle Detection	Node Existence	Node Count	Degree Count	Average
Small-Scale (40 Nodes)											
Text-based	GPT-4o [30]	100	22.2	20.6	25.8	77.2	78.2	100	84.2	29.8	59.8
	Claude-3.7 [2]	99.2	22.4	21.0	21.2	79.8	81.8	100	80.2	32.2	59.8
	DeepSeek-R1 [8]	100	32.6	26.6	30.8	86.6	86.8	100	<u>90.4</u>	31.0	65.0
Tool-based	Graph-TF [43]	48.0	75.6	71.0	63.8	44.2	66.8	50.2	77.2	50.0	60.8
	GraphTool-Ins [39]	94.2	98.0	97.8	98.2	96.8	<u>99.0</u>	100	100	97.8	98.0
	GPT4o-FC [30]	95.0	99.2	<u>98.4</u>	99.0	99.2	100	100	100	<u>99.0</u>	<u>98.9</u>
Code-based	GraphTeam [23]	86.6	84.8	82.8	80.2	97.6	80.0	96.8	70.8	97.6	86.4
	GCoder [44]	82.0	84.6	83.4	82.8	90.6	80.6	88.0	74.0	87.8	83.8
Ours	GraphCogent	<u>99.4</u>	<u>98.2</u>	99.0	<u>98.6</u>	<u>97.8</u>	100	<u>99.2</u>	100	100	99.1
Middle-Scale (100 Nodes)											
Text-based	GPT-4o [30]	90.4	6.8	5.4	7.2	54.0	67.6	99.8	58.6	6.8	44.1
	Claude-3.7 [2]	93.0	6.2	4.8	6.8	55.8	73.0	99.2	52.2	7.2	44.2
	DeepSeek-R1 [8]	<u>98.0</u>	7.4	6.2	8.0	60.6	76.6	100	60.4	7.4	47.2
Tool-based	Graph-TF [43]	43.2	68.0	63.8	57.4	39.8	60.0	45.2	69.4	45.0	54.6
	GraphTool-Ins [39]	84.8	89.0	88.0	88.4	89.0	89.0	90.6	92.0	88.0	88.8
	GPT4o-FC [30]	85.4	<u>89.2</u>	<u>88.6</u>	<u>89.0</u>	<u>89.2</u>	<u>89.4</u>	97.0	<u>94.2</u>	89.0	<u>90.1</u>
Code-based	GraphTeam [23]	91.6	80.2	73.0	65.8	93.2	69.6	89.6	65.4	<u>95.2</u>	80.4
	GCoder [44]	80.0	68.0	60.8	68.2	82.4	71.4	78.2	67.4	82.8	73.2
Ours	GraphCogent	98.2	97.8	98.8	98.2	97.4	99.0	<u>99.4</u>	97.6	99.8	98.5
Large-Scale (1000 Nodes)											
Ours	GraphCogent	97.0	97.4	98.6	98.0	97.2	97.0	96.8	97.2	99.6	97.6

Table 2: Performance evaluation on out-toolset tasks on middle-scale graphs. Note: Except for the traffic prediction (which values represent MAE), all other values represent Accuracy.

Method	Web			Transportation			Social			Citation		
	Common Neighbor	PageRank Calcu.	Link Pred.	Max Flow	Diameter Calcu.	<i>Traffic Pred.</i>	Maxcore Calcu.	Connect. Compo.	Link Pred.	Reference Match	Cluster. Coeff.	Node Class.
GraphTeam	51.2	76.8	71.6	72.4	61.6	96.7	69.8	72.0	76.4	78.8	76.4	72.0
GCoder	49.0	74.0	68.5	70.0	58.8	99.2	67.2	69.0	72.0	75.0	74.0	68.0
GraphCogent	88.8	92.6	86.4	89.6	92.4	35.1	91.6	87.6	85.2	90.0	85.2	89.6

Table 3: Reasoning token comparison with code-based baselines. Note: Input and Output are abbreviations for average input tokens and average output tokens respectively.

Method	In-Toolset Tasks						Out-Toolset Tasks					
	Shortest Path			Cycle Detection			Max Flow			Link Prediction		
	Input	Output	Accuracy	Input	Output	Accuracy	Input	Output	Accuracy	Input	Output	Accuracy
GraphTeam	4248	3285	65.8	4207	2901	69.6	4122	3023	72.4	4423	3820	76.4
GCoder	3782	1051	68.2	3640	620	71.4	3927	1181	70.0	3855	1909	72.0
GraphCogent	2794	615	98.2	2533	595	99.0	3009	1044	89.6	3128	1244	90.2

requiring multi-step reasoning (e.g., only 30.8% accuracy on shortest path), and this issue becomes more pronounced as the scale increases. Tool-based approaches such as GPT4o-FC demonstrates robust performance across all tasks on small-scale graphs by leveraging GPT4o’s reasoning capabilities. However, they suffer 10% degradation when scaling from 40 to 100 nodes. The code-based GraphTeam and GCoder demonstrate limitations in code generation across multiple tasks. The complex data preprocessing and reasoning requirements overload the model’s working memory, resulting in lower performance. In contrast, our method maintains state-of-the-art performance across all scales with less variance.

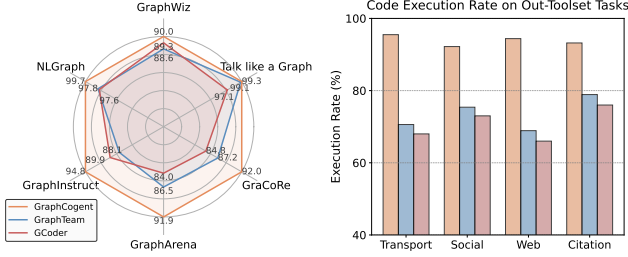
Notably, while no baseline could handle large-scale graphs, we have only included the results of our approach.

5.2.2 Out-of-Toolset Performance. In Table 2, we compare GraphCogent with GraphTeam and GCoder on out-toolset tasks. Focusing on methods capable of handling uncovered tasks, we exclude tool-based methods since they cannot handle uncovered tasks. Our method demonstrates superior adaptability, achieving 20% improvements over GraphTeam and GCoder across four domains.

We compare code-based approaches on public benchmarks (left part of Fig. 5) using the shared task intersection for fair evaluation. GraphCogent performs well across all six public datasets.

Table 4: Reasoning time comparison between GraphCogent and other baselines.

Method	Shortest Path		Cycle Detection		Triangle Count		Path Existence		Link Prediction	
	Time	Accuracy	Time	Accuracy	Time	Accuracy	Time	Accuracy	Time	Accuracy
GPT4o-Function Calling	12.2s	89.0	10.7s	89.4	11.3s	88.6	12.9s	89.2	/	/
Graph-Toolformer	18.1s	57.4	16.6s	60.0	18.9s	63.8	17.3s	39.8	/	/
GraphTool-Instruction	19.8s	88.4	18.3s	89.0	20.1s	88.0	19.2s	89.0	/	/
GraphTeam	58.1s	65.8	56.6s	69.6	57.3s	73.0	58.9s	93.2	59.5s	76.4
GCoder	21.5s	68.2	17.5s	71.4	17.4s	60.8	13.3s	82.4	27.7s	72.0
GraphCogent	22.8s	98.2	23.3s	99.0	20.1s	98.8	19.2s	97.4	24.0s	85.2

**Figure 5: Average performance on public benchmarks (left). Code execution rate on out-toolset tasks (right).**

We attribute this to our working-memory-inspired pipeline, which reduces execution errors and improves efficiency. The Code Execution Rate results (right part of Fig. 5) further show higher one-pass success than GraphTeam’s three-retry strategy and GCoder’s compiler-feedback reinforcement tuning. This aligns with our working memory analysis: when an LLM must retain graph topology, perform multi-step reasoning, and generate executable code simultaneously, both accuracy and executability degrade.

5.2.3 Efficiency and Scalability Across Models. Experiments on effectiveness evaluation of our approach demonstrate the superiority in two key aspects. First, our token consumption analysis in Table 3 shows consistent improvements across all tasks, with significantly reduced input token consumption and substantial output token savings over GraphTeam (approximately 80% for in-toolset tasks and 30% for out-toolset tasks) while maintaining high accuracy. Second, we compare Tool-based and Code-based methods on reasoning time in Table 4. While GPT4o-FC’s API implementation demonstrates significant time advantages over local open-source LLMs, our local method GraphCogent achieves competitive speed while maintaining superior accuracy. The additional reasoning time in our framework mainly stems from multi-round sampling in the Sensory module. In contrast, although GraphTeam (based on GPT4o-mini) benefits from faster API inference, its performance is compromised by working memory overload in complex tasks and the overhead of its three-retry mechanism, making its direct code generation approach both slower and less accurate.

Although our experiments primarily focus on the Llama3.1-8B, the framework’s modular design ensures compatibility with both open-source and closed-source LLMs. Table 5 reveals our framework’s adaptability. For open-source models like GLM4-9B-chat [11] and Qwen2.5-7B-coder-Instruct [31], our framework significantly enhances their reasoning capabilities especially for complex tasks requiring multi-step graph reasoning, improving performance from near-zero accuracy in tasks such as Shortest Path

Table 5: Adaptability across different LLMs. Values are GraphCogent accuracy with gains over naive baselines.

Base Model	Shortest Path	Cycle Detect.	Max Flow	Link Pred.
GLM4-9B	96.2 (+96.2)	96.8 (+28.4)	81.4 (+81.4)	82.2 (+33.6)
Qwen2.5-7B	92.2 (+92.2)	90.4 (+34.0)	91.2 (+91.2)	87.2 (+36.2)
GPT-4o	100.0 (+92.8)	100.0 (+32.4)	99.2 (+89.0)	90.8 (+28.4)
DeepSeek-R1	100.0 (+92.0)	100.0 (+23.3)	99.2 (+89.6)	91.2 (+30.8)

Table 6: Accuracy on larger graph.

Scale	Shortest Path	Cycle Detect.	Triangle Count	Path Exist.	PageRank Calcu.	Max Flow
2000	97.0	97.0	98.6	97.0	93.0	89.6
5000	98.2	97.0	98.8	98.0	90.8	87.8
10000	97.4	96.8	99.0	97.0	89.8	88.4

and Max Flow to high accuracy. Second, based on the framework’s instruction without training, massive-scale LLMs like GPT-4o and DeepSeek-R1 achieve high performance on both in-toolset tasks and out-toolset, demonstrating the universal applicability of our framework design. The scalability evaluation in Table 6 on larger graphs further demonstrates the effectiveness of our framework, where it maintains stable performance across various tasks. Through ablation studies in Appendix B, we also examine the contributions of each component.

6 Conclusion

In this work, we proposed GraphCogent, a novel agent framework that enhances LLMs’ graph reasoning capabilities by mitigating their working memory constraints. Inspired by human cognitive architecture, GraphCogent integrates three key modules: Sensory Module (standardizing diverse graph text representations), Buffer Module (integrating and indexing graph data), and Execution Module (combining tool-calling and tool-creation reasoning). To evaluate LLMs on real-world graph reasoning tasks, we introduce Graph4real, a benchmark featuring large-scale graphs for four real-world domains. Experiments show that GraphCogent achieves 20% higher than existing state-of-the-art code-based methods while reducing over 30% token usage.

Acknowledgments

This work is supported by National Natural Science Foundation of China No.62406057, the Fundamental Research Funds for the Central Universities No.ZYGX2025XJ042, the Noncommunicable Chronic Diseases-National Science and Technology Major Project No.2023ZD0501806, and the Sichuan Science and Technology Program under Grant No.2024ZDZX0011.

References

- [1] Mary Jean Amon and Bennett I. Bertenthal. 2018. Auditory Versus Visual Stimulus Effects on Cognitive Performance During the N-back Task. In *CogSci*. Madison, Wisconsin, USA, 1304–1309.
- [2] Anthropic. 2024. The Claude 3 Model Family: Opus, Sonnet, Haiku. In *Claude-3 Model Card*.
- [3] Alan Baddeley and Graham Hitch. 1974. Working Memory. In *Psychology of Learning and Motivation*. Vol. 8. Elsevier, 47–89.
- [4] Alan Baddeley, Christopher Jarrold, and Faraneh Vargha-Khadem. 2011. Working Memory and the Hippocampus. *Journal of Cognitive Neuroscience* 23, 12 (2011), 3855–3861.
- [5] Richard Bellman. 1958. On a routing problem. *Quart. Appl. Math.* 16, 1 (1958), 87–90.
- [6] Chao Chen, Karl Petty, Alexander Skabardonis, Pravin Varaiya, and Zhanfeng Jia. 2001. Freeway Performance Measurement System: Mining Loop Detector Data. *Transportation Research Record* 1748, 1 (2001), 96–102.
- [7] Nuo Chen, Yuhua Li, Jianheng Tang, and Jia Li. 2024. GraphWiz: An Instruction-Following Language Model for Graph Computational Problems. In *KDD*. Barcelona, Spain, 353–364.
- [8] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *CoRR* abs/2501.12948 (2025).
- [9] Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. 2024. Talk like a Graph: Encoding Graphs for Large Language Models. In *ICLR*. Vienna, Austria. *CoRR* abs/2312.11805 (2023).
- [10] Gemini Team. 2023. Gemini: A Family of Highly Capable Multimodal Models. *CoRR* abs/2312.11805 (2023).
- [11] Team GLM. 2024. ChatGLM: A Family of Large Language Models from GLM-130B to GLM-4 All Tools. *CoRR* abs/2406.12793 (2024).
- [12] Dongyu Gong, Xingchen Wan, and Dingmin Wang. 2024. Working Memory Capacity of ChatGPT: An Empirical Study. In *AAAI* 10048–10056.
- [13] Jiayan Guo, Lun Du, and Hengyu Liu. 2023. GPT4Graph: Can Large Language Models Understand Graph Structured Data? An Empirical Evaluation and Benchmarking. *CoRR* abs/2305.15066 (2023).
- [14] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *ICLR*. Virtual Event.
- [15] Susanne M. Jaeggi, Martin Buschkuhl, John Jonides, and Walter J. Perrig. 2008. Improving fluid intelligence with training on working memory. *Proceedings of the National Academy of Sciences* 105, 19 (2008), 6829–6833.
- [16] Susanne M. Jaeggi, Martin Buschkuhl, Walter J. Perrig, and Beat Meier. 2010. The concurrent validity of the N-back task as a working memory measure. *Memory* 18, 4 (2010), 394–412.
- [17] Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Xin Zhao, and Ji-Rong Wen. 2023. StructGPT: A General Framework for Large Language Model to Reason over Structured Data. In *EMNLP*. ACL, Singapore, 9237–9251.
- [18] Wayne K. Kirchner. 1958. Age differences in short-term retention of rapidly changing information. *Journal of Experimental Psychology* 55, 4 (1958), 352–358.
- [19] Roberta L. Klatzky, Nicholas A. Giudice, James R. Marston, Jerome Tietz, Reginald G. Golledge, and Jack M. Loomis. 2008. An n-back task using vibrotactile stimulation with comparison to an auditory analogue. *Behavior Research Methods* 40, 1 (2008), 367–372.
- [20] Jure Leskovec, Kevin J. Lang, Anirban Dasgupta, and Michael W. Mahoney. 2009. Community Structure in Large Networks: Natural Cluster Sizes and the Absence of Large Well-Defined Clusters. *Internet Mathematics* 6, 1 (2009), 29–123.
- [21] Junchen Li, Rongzheng Wang, Yihong Huang, Qizhi Chen, Jiasheng Zhang, and Shuang Liang. 2025. NeuroPath: Neurobiology-Inspired Path Tracking and Reflection for Semantically Coherent Retrieval. *CoRR* abs/2511.14096 (2025).
- [22] Jiakai Li, Rongzheng Wang, Yizhuo Ma, Shuang Liang, Guangchun Luo, and Ke Qin. 2025. DSAS: A Universal Plug-and-Play Framework for Attention Optimization in Multi-Document Question Answering. *CoRR* abs/2510.12251 (2025).
- [23] Xin Li, Qizhi Chu, Yubin Chen, Yang Liu, Yaoqi Liu, Zekai Yu, Weize Chen, Chen Qian, Chuan Shi, and Cheng Yang. 2024. GraphTeam: Facilitating Large Language Model-based Graph Analysis via Multi-Agent Collaboration. *CoRR* abs/2410.18032 (2024).
- [24] Shaoyu Liu, Jianing Li, Guanghui Zhao, Yunjian Zhang, Xin Meng, Fei Richard Yu, Xiangyang Ji, and Ming Li. 2025. EventGPT: Event Stream Understanding with Multimodal Large Language Models. In *CVPR*. Nashville, TN, USA, 29139–29149.
- [25] Zihan Luo, Xiran Song, Hong Huang, Jianxun Lian, Chenhao Zhang, Jinqi Jiang, and Xing Xie. 2024. GraphInstruct: Empowering Large Language Models with Graph Understanding and Reasoning Capability. *CoRR* abs/2403.04483 (2024).
- [26] Yizhuo Ma, Rongzheng Wang, Shuang Liang, Guangchun Luo, and Ke Qin. 2025. TopoLLM: LLM-driven adaptive tool learning for real-time emergency network topology planning. *Digital Communications and Networks* (2025).
- [27] Julian J. McAuley and Jure Leskovec. 2012. Learning to Discover Social Circles in Ego Networks. In *NeurIPS*. Lake Tahoe, Nevada, United States, 548–556.
- [28] George A. Miller. 1956. The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological Review* 63, 2 (1956), 81–97.
- [29] William O'Donohue, Kyle E. Ferguson, and Amy E. Naugle. 2003. The structure of the cognitive revolution: an examination from the philosophy of science. *The Behavior Analyst* 26, 1 (2003), 85–110.
- [30] OpenAI. 2023. GPT-4 Technical Report. *CoRR* abs/2303.08774 (2023).
- [31] Qwen Team. 2024. Qwen2.5 Technical Report. *CoRR* abs/2412.15115 (2024).
- [32] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *NeurIPS*. New Orleans, LA, USA.
- [33] Yufei Shi, Weilong Yan, Gang Xu, Yumeng Li, Yuchen Li, Zhenxi Li, Fei Richard Yu, Ming Li, and Si Yong Yeo. 2025. PVChat: Personalized Video Chat with One-Shot Learning. *CoRR* abs/2503.17069 (2025).
- [34] Songbai Tan, Xuerui Qiu, Yao Shu, Gang Xu, Linrui Xu, Xiangyu Xu, Huiping Zhuang, Ming Li, and Fei Richard Yu. 2025. WMarkGPT: Watermarked Image Understanding via Multimodal Large Language Models. In *ICML*. Vancouver, BC, Canada.
- [35] Jianheng Tang, Qifan Zhang, Yuhua Li, Nuo Chen, and Jia Li. 2025. GraphArena: Evaluating and Exploring Large Language Models on Graph Computation. In *ICLR*. Singapore.
- [36] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *CoRR* abs/2302.13971 (2023).
- [37] Heng Wang, Shangbin Feng, Tianxing He, Zhaoxuan Tan, Xiaochuang Han, and Yulia Tsvetkov. 2023. Can Language Models Solve Graph Problems in Natural Language?. In *NeurIPS*. New Orleans, LA, USA.
- [38] Jianing Wang, Junda Wu, Yupeng Hou, Yao Liu, Ming Gao, and Julian J. McAuley. 2024. InstructGraph: Boosting Large Language Models via Graph-centric Instruction Tuning and Preference Alignment. In *ACL*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.), 13492–13510.
- [39] Rongzheng Wang, Shuang Liang, Qizhi Chen, Jiasheng Zhang, and Ke Qin. 2025. GraphTool-Instruction: Revolutionizing Graph Reasoning in LLMs through Decomposed Subtask Instruction. In *KDD*, 1492–1503.
- [40] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *NeurIPS*. New Orleans, LA, USA.
- [41] Zhilin Yang, William W. Cohen, and Ruslan Salakhutdinov. 2016. Revisiting Semi-Supervised Learning with Graph Embeddings. In *ICML*, Vol. 48. New York City, USA, 40–48.
- [42] Zike Yuan, Ming Liu, Hui Wang, and Bing Qin. 2025. GraCoRe: Benchmarking Graph Comprehension and Complex Reasoning in Large Language Models. In *COLING*. Abu Dhabi, UAE, 7925–7948.
- [43] Jiawei Zhang. 2023. Graph-ToolFormer: To Empower LLMs with Graph Reasoning Ability via Prompt Augmented by ChatGPT. *CoRR* abs/2304.11116 (2023).
- [44] Qifan Zhang, Xiaobin Hong, Jianheng Tang, Nuo Chen, Yuhua Li, Wenzhong Li, Jing Tang, and Jia Li. 2024. GCoder: Improving Large Language Model for Generalized Graph Problem Solving. *CoRR* abs/2410.19084 (2024).

A Additional Experimental Settings

Training. We use Llama3.1-8B as the backbone for all agents, which are trained on Graph4real. We use 4 of 20 prompt templates for training, and exclude their corresponding tasks from the test set.

Hyperparameters. For capability-aware negatives, we set $K = 8$ and use $\tau_{\text{high}} = 0.55$, $\tau_{\text{low}} = 0.45$. We train with Llama-Factory. Instruction fine-tuning uses $\text{lr} \times 10^{-5}$, warmup 0.1, batch size 4, cosine scheduler, and 5 epochs. CMPO uses $\text{lr} \times 10^{-6}$, warmup 0.1, batch size 1 with 4-step gradient accumulation, cosine scheduler, and 5 epochs.

Setup. Open-source methods are trained on 8×NVIDIA A800 (80G) and evaluated on 8×NVIDIA RTX 4090 (24G). We use temperature=0.7 and top_p=1 for inference. Closed-source LLMs are accessed via official APIs with the same decoding settings.

B Ablation Study

We conduct ablation studies to evaluate each module and its key components through the following research questions:

Table 7: Impact of Graph Verifier on transformation reliability and global consistency. Edit Distance is computed by summing the edit distances of all windows within each graph.

Representation Type	Edit Dist. (Before)	Edit Dist. (After)	GED	BER (%)	Verifier Triggers
Adjacency List	44.8 $\pm$ 7.6	5.2 $\pm$ 1.1	0.8 $\pm$ 0.2	99.8 $\pm$ 0.06	6.2 $\pm$ 1.0
Symbolic Notation	57.3 $\pm$ 8.9	5.9 $\pm$ 1.2	1.1 $\pm$ 0.4	99.7 $\pm$ 0.08	9.0 $\pm$ 1.3
Linguistic Description	65.9 $\pm$ 9.1	5.1 $\pm$ 1.0	1.0 $\pm$ 0.3	99.8 $\pm$ 0.07	9.1 $\pm$ 1.2

B.1 Ablation Studies on Sensory Module

RQ1: How does the Graph Verifier component enhance transformation reliability and ensure global consistency?

As shown in Fig. 3b even based on GEC optimization, the inherent uncertainty in LLM outputs can still introduce parsing inaccuracies during graph transformation. Table 7 further demonstrates the critical role of the Graph Verifier in ensuring reliable graph transformation and global consistency.

We randomly sample 100 graphs (each containing 100 nodes with 300 edges) from PeMS dataset, partitioning each graph into 50-edge subsets with different graph text representations (i.e., adjacency list, symbolic notation, and linguistic description). We evaluate edit distance during transformation and assess post-merge global consistency using two criteria: (i) Global Edit Distance (GED), the edit distance between the original graph G and the merged graph $\hat{G}$; and (ii) Boundary–Edge Recall (BER), the recall of edges whose endpoints lie in different windows or touch a window boundary. The results show that introducing the Graph Verifier reduces structural errors by approximately 90%, while maintaining operational efficiency with fewer than 10 verification triggers. This retry frequency (occurring in approximately 600 LLM calls) remains acceptable while significantly reducing edit distance.

B.2 Ablation Studies on Buffer Module

RQ2: How does the Buffer Mechanism enhance the framework’s reasoning capability?

We select four tasks within Graph4real on middle-scale. The experimental results in Table 8 show that removing the Buffer Module causes substantial performance decreases across all tasks. This result evidences the Buffer Module’s critical role in addressing LLMs’ working memory constraints for graph reasoning. Without the Buffer Module, when the graph scale exceeds the capacity of LLMs’ working memory, the LLMs exhibit substantial information loss regarding graph topology.

Table 8: Performance impact of Buffer Module removal.

Variants	Shortest Path	Cycle Detect.	Max Flow	Link Pred.
w/ Buffer	98.2	99.0	89.6	85.2
w/o Buffer	68.4	69.2	53.8	54.2

B.3 Ablation Studies on Execution Module

RQ3: Does Thinking Policy Initialization (TPI) improve Execution Agent’s tool selection?

We select four tasks within Graph4real on middle-scale. Three versions of the Execution Agent are evaluated: the original untuned agent (Llama3.1-8B), agent fine-tuned using the Naive SFT method,

Table 9: Impact of Thinking Policy Initialization on Execution Agent performance.

Training Method	Shortest Path	Cycle Detect.	Max Flow	Link Pred.
Llama3.1-8B (Base)	78.4	79.2	35.8	36.2
Llama3.1-8B w/ SFT	89.3	90.2	43.8	44.2
Llama3.1-8B w/ TPI	98.2	99.0	70.2	65.0

and agent fine-tuned with our Thinking Policy Initialization (TPI). The Naive SFT approach involves training Llama3.1-8B by directly mapping problem statements to their outputs, where each output consists of a brief analytical statement containing the tool selection result. As shown in Table 9, the TPI method shows consistent performance advantages across different tasks. For in-toolset tasks, TPI achieves a 5% performance improvement over the naive approach, as the explicit reasoning chains enable the agent to better understand task intent and select appropriate tools. Through analysis of the reasoning process, we observe that compared to Naive SFT’s tendency to directly output corresponding tools (i.e., simply fitting tool results), the TPI teaches the agent to comprehend task intentions and demonstrates reasonable reasoning paths, leading to correct tool selection. This advantage becomes even more significant for out-toolset tasks, where the TPI shows substantially better generalization capabilities than naive method.

Table 10: Impact of CMPO on Execution Agent.

Variants	Shortest Path	Cycle Detect.	Max Flow	Link Pred.
w/ CMPO	98.2	99.0	89.7	85.2
w/o CMPO	97.2	97.8	70.2	64.8

However, the TPI still has limitations. Through analyzing the reasoning results for out-toolset tasks like Max Flow and Link Prediction, we identify a tendency of the LLM to forcibly associate tools. For instance, although no single tool can solve the Max Flow problem, unlike the Naive SFT-tuned LLM, which randomly generates a tool or calls a non-existent tool, the TPI-tuned LLM attempts to combine tools to derive a solution. This preference actually contradicts practical decision-making. In this task, we would prefer the agent to directly generate the task-specific tool of Max Flow through Tool Creator rather than repeatedly interact with tools of Execution Agent. This observation motivates our implementation of CMPO alignment to refine the Execution Agent’s preferences.

RQ4: Why employ Capability-Margin Preference Optimization (CMPO) for tool capability discrimination?

Building upon the findings from B.3, the core objective of CMPO alignment is to refine the Execution Agent’s tool selection preferences for improving toolset discrimination capabilities. Although tasks are not strictly binary (i.e., solvable either by tools or not), the

Table 11: Impact of Tool Creation Fine-tuning.

Task	Without SFT		With Code SFT		With Tool SFT	
	Acc.(%)	Execut.(%)	Acc.(%)	Execut.(%)	Acc.(%)	Execut.(%)
Max Flow	38.2	58.4	72.2	81.2	89.6	96.3
Common Neighbors	42.6	63.1	70.6	78.3	88.8	95.8
Link Prediction	39.8	61.6	58.6	65.2	85.2	88.4
Node Classification	41.2	59.8	60.8	64.7	89.6	89.9

LLM requires a decision-making mechanism to optimize resource efficiency. The goal is to minimize resource consumption, single tool calls can significantly reduce the LLM's interaction tokens. The experimental results in Table 10 demonstrate that CMPO not only preserves Execution Agent's in-toolset selection performance but also substantially enhances out-toolset discrimination capability.

RQ5: How does Tool Creation Fine-tuning improve Tool Creator's generation capability?

We select four out-toolset tasks within Graph4real on middle-scale. Three versions of the Tool Creator are evaluated: the original untuned agent (Llama3.1-8B), agent fine-tuned using Code-Enhanced SFT method, and agent fine-tuned using Tool Creation Fine-tuning method. Table 11 reports the accuracy and executable percentage. The improvements in both accuracy and executability metrics confirm the effectiveness of Tool Creation Fine-tuning. Compared to Code-Enhanced SFT, especially for mainstream open-source LLMs, it is more challenging to enable LLMs to master the reasoning capabilities for complex tasks through code training. The complexity of the tasks and the limitations of LLM capabilities hinder their generalization performance on complex tasks. In contrast, Tool Creation Fine-tuning effectively enhances accuracy and executability by allowing the agent to focus more on generating core task-specific tools.

C Edit distance and Cost for GEC

C.1 Edit distance

Real-world scenarios present graphs in multiple text forms—adjacency lists, symbolic notations, and linguistic descriptions—making rule-based preprocessing (e.g., regular expressions) impractical for standardization. Following human external sensory processing, our Sensory Module converts these varied inputs into a standardized adjacency-list representation. Because small transformation errors can propagate downstream reasoning on topology-rich graphs, we perform rigorous quality assessment using edge edit distance (Algorithm 1), which measures the minimum edge additions and deletions needed to transform one graph into another.

C.2 Cost for Graph Efficiency Coefficient

We select token count as GEC's cost metric for three reasons: (i) token-based pricing is the universal billing standard across LLM platforms (GPT, Claude, DeepSeek), enabling fair comparison; (ii) in tests on 100 graphs (300 edges) with four granularities, processing time varied by less than 5%, since transformation outputs are length-stable, so time is not discriminative; (iii) the dominant expense is input tokens from the fixed instruction used to standardize graphs. We therefore use normalized token counts to capture the

granularity trade-off: finer sampling increases LLM calls and accumulates instruction tokens, whereas coarser sampling reduces calls at the risk of topological distortion.

Algorithm 1 Graph Edit Distance Computation

Require: Source graph $G = (V, E)$; Target graph $H = (V, E')$ with identical node set V

Ensure: Edit distance $d_{\text{edit}}(G, H)$

```

1: Assert  $V(H) = V(G)$ 
2:  $E_G \leftarrow \text{set}(E)$ ,  $E_H \leftarrow \text{set}(E')$ 
3: if  $E_G = E_H$  then
4:   return 0
5: end if
6:  $\mathcal{A} \leftarrow E_H \setminus E_G$ ,  $\mathcal{R} \leftarrow E_G \setminus E_H$ 
7:  $a \leftarrow |\mathcal{A}|$ ,  $r \leftarrow |\mathcal{R}|$ 
8:  $w_{\text{add}} \leftarrow 1$ ,  $w_{\text{rem}} \leftarrow 1$ 
9:  $d_{\text{edit}}(G, H) \leftarrow w_{\text{add}} \cdot a + w_{\text{rem}} \cdot r$ 
10: return  $d_{\text{edit}}(G, H)$ 

```

D Error Analysis

In this section, we analyze the errors of GraphCogent within the Graph4real benchmark for both in-toolset and out-toolset tasks.

D.1 In-toolset tasks

We select four in-toolset tasks and analyze error patterns across 500 test instances at the middle scale (100 nodes). The errors are categorized into two types: (i) *Edge Error*: Structural deviations between the original graph and the transformed graph caused by inconsistent edges during the transformation. (ii) *Tool Error*: Incorrect tool selection or parameterization in the Execution Module.

The results in Table 12 demonstrate that GraphCogent achieves high accuracy across in-toolset tasks, but performance varies by task type. For edge-dependent tasks (e.g., Edge Count, Triangle Count), errors primarily stem from Edge Errors, structural discrepancies introduced during graph transformation. These tasks rely on precise edge information, making them sensitive to minor deviations in the adjacency list conversion. In contrast, topology-independent tasks (e.g., Cycle Detection, Path Existence) show a reversed pattern. While Edge Errors may occur, their impact is mitigated because task outcomes do not depend on specific edge subsets. Therefore, Tool Errors dominate the error types for these tasks.

D.2 Out-toolset tasks

For out-toolset tasks, we select Max Flow on Transportation and Link Prediction on Social, analyzing error patterns across 500 test instances at the middle-scale graph (100 nodes). We classify errors

Table 12: Error distribution across in-toolset tasks.

Task	Edge	Tool	Accuracy (%)
Edge Count	19	2	97.8
Triangle Count	18	1	98.8
Cycle Detection	21	5	99.0
Path Existence	19	13	97.4

into three categories: (i) *Discrimination Error*: Occurs when the Execution Agent fails to identify the task as an out-toolset task, leading to inappropriate tool calls instead of activating the Tool Creator. (ii) *Execution Error*: Arises when the task-specific tool generated by the Tool Creator contains logical flaws or implementation issues that prevent successful execution. (iii) *Format Error*: Arises when the task-specific tool produces correct results but returns them in non-compliant formats.

The results are reported in Table 13. For Max Flow, discrimination errors dominate, primarily due to the Execution Agent’s failure to discriminate task boundaries when problems exceed the toolset’s coverage. In such cases, the Execution Agent incorrectly attempts tool-based solutions despite the task requiring tool creation. Format errors are secondary, often manifesting as mismatched outputs (e.g., returning paths instead of the max flow value). In contrast, Link Prediction is primarily affected by execution errors, which stem from internal tensor mismatches in the generated neural models (e.g., dimension inconsistencies in GCN layers or invalid adjacency matrix operations). Both discrimination and format errors occur less frequently, suggesting that the model reliably identifies the task type and produces valid outputs.

Table 13: Error distribution across out-toolset tasks.

Task	Discrimination	Execution	Format	Accuracy (%)
Max Flow	35	6	11	89.6
Link Prediction	6	58	10	85.2

E Graph Reasoning Task Definition

We define 21 tasks across four domains (Web, Social, Transportation, Citation).

Classes: (1) Graph Structural Querying tests constant-time lookups with five tasks: Edge Count, Node Count, Degree Count, Edge Existence, and Node Existence. All are in-toolset with direct tools over the common-toolset. (2) Graph Algorithmic Reasoning measures multi-step reasoning capability. In-toolset: Cycle Detection, Triangle Count, Path Existence, and Shortest Path. Out-toolset: Maximum Flow, Diameter Calculation, Max-Core Calculation, Connected Components, Common Neighbors, PageRank, Reference

Match, and Clustering Coefficient; these are domain-specialized and span linear to cubic complexity. (3) Graph Predictive Modeling evaluates neural prediction: Traffic Prediction (Transportation), Link Prediction (Social, Web), and Node Classification (Citation), each with a fixed reference model and standardized preprocessing and metrics.

Task generation: per domain we enumerate five real scenarios and bind them to 20 prompt templates. DeepSeek-R1 instantiates tasks under domain constraints. Validation combines automated screening with DeepSeek-R1 and human review. The final dataset contains 4,200 questions across domains and task classes.

F GraphCogent Workflow

Algorithm 2 presents the workflow of GraphCogent.

Algorithm 2 GraphCogent Framework

Require: Graph task T , Input graph G

Ensure: Task result R

```

1: Sensory Module:
2: if  $G$  is file-based (edgelist/txt/json) then
3:   Bypass transformation; forward  $G$  directly to Buffer
4: else
5:   Apply sliding-window sampling (GEC-selected; default 50
   edges with 5% overlap)
6:   Transform text to adjacency lists via heuristic prompts
7:   Verify via Graph Verifier (quantity consistency + format
   compliance)
8: end if
9: Buffer Module:
10: Construct complete graph from adjacency lists
11: Transform to multiple representations: NetworkX/PyG/NumPy
12: Build data indices (dimensionality, schema, metadata)
13: Store preprocessed graph data  $\tilde{G}$ 
14: Execution Module:
15: if  $T$  is in common toolset coverage then
16:   Select appropriate tool from common toolset
17:   Retrieve required data format from Buffer Module
18:   Execute tool calling on  $\tilde{G}$ ; return  $R$ 
19: else
20:   Generate task-specific model  $M$  for  $T$ 
21:   Combine  $M$  with  $\tilde{G}$  from Buffer Module
22:   Execute generated model; return  $R$ 
23: end if

```
